Q2 Big Integer – Solutions

DSAPS (M26-CS 1.304) · Assignment 3 · In-lab question 2

In the lab, everyone solved two of these three problems. All three are built on the same small library of big-integer operations, so this document starts with that library and then explains each problem. For every problem you get the idea, a worked example, the reference code, its running time, and the **mistakes we actually saw in submissions** – check your own code against those.

Contents

0. The big-integer core: compare, add, subtract, multiply, divide
1. Problem 1 – Custom Precedence Calculator
2. Problem 2 – Big Integer Square Root
3. Problem 3 – Catalan Number

Each complete solution file is: the `#include` lines allowed by that problem, then the core from Section 0, then the problem-specific code shown in that problem's section. All three reference solutions pass every hidden test, also when compiled with AddressSanitizer and UndefinedBehaviorSanitizer.

Notation

n and m are the numbers of digits of the two operands. A number is stored as a `string` of decimal digits, most significant digit first: 12345 is `"12345"`. All values in these problems are non-negative.

0 · The big-integer core

Operation	Function	Idea	Cost
Compare	`compareBig`	More digits = bigger; same length → first differing digit decides	O(n + m)
Add	`addBig`	Right to left, digit + digit + carry	O(n + m)
Subtract (a ≥ b)	`subBig`	Right to left, digit − digit − borrow	O(n + m)
Multiply	`mulBig`	School multiplication: every digit pair, added at its place value	O(n · m)
Divide (integer)	`divBig`	Long division: bring down a digit, subtract the divisor ≤ 9 times	O(n · m)

Two rules that prevent most bugs

1. **Zero must stay `"0"`, never `""`.** When removing leading zeros, stop one character before the end: `while (i + 1 < s.size() && s[i] == '0')`. Several submissions removed *every* zero, so `3/7` printed an empty line instead of `0`.
2. **Build strings by appending, then reverse once.** Writing `ans = digit + ans` copies the whole string for every digit, which turns an O(n) addition into O(n^2). Inside a division (thousands of subtractions) this was the difference between 0.2 s and several seconds.

Multiplication: where does each digit product go?

Digit `a[i]` stands for `a[i]` × 10^{n-1-i} (the index counts from the left, place value from the right). Likewise `b[j]` stands for `b[j]` × 10^{m-1-j}. Multiplying adds the exponents, so the product `a[i]*b[j]` belongs at place 10^{pos} with

```
pos = (n - 1 - i) + (m - 1 - j)
```

The core stores the result *least* significant digit first (`res[k]` = digit at 10^k), so the product is added straight into `res[pos]` and any carry moves to `pos + 1`. For 123 × 45 the three rows add 3·45 = 135, then 20·45 = 900, then 100·45 = 4500, giving 5535.

The common alternative `res[i + j + 1] += a[i]*b[j]` is the same formula with `res` stored most-significant-first: index = (n + m − 1) − pos = i + j + 1. Both are correct.

Division: long division, digit by digit

Bring down one digit of *a* at a time and count how many times *b* fits into the running remainder. Example 7391 ÷ 23:

Bring down	Remainder	Subtract 23 …	Quotient digit	Left over
7	7	0 times	0	7
3	73	73 → 50 → 27 → 4	3	4
9	49	49 → 26 → 3	2	3
1	31	31 → 8	1	8

Quotient `0321` → **321**, remainder 8 (23 × 321 + 8 = 7391). Before a digit is brought down the remainder is < b, so afterwards it is < 10·b: **at most 9 subtractions per digit**, i.e. O(9·n·m) = O(n·m) in total.

Don't divide by repeated subtraction of the whole number

"Subtract b from a until it no longer fits and count" needs as many steps as the *value* of the quotient. For a 400-digit number divided by 3 that is about 10^{399} steps – it never finishes. Long division needs at most 9 per digit. The hidden tests were designed so that this version times out.

Code: shared core

```
// ---- Big integer core (shared by all three solutions) ----------------------
// Non-negative integers stored as decimal strings, most significant digit first.

// Remove leading zeros, but always keep at least one digit: "007" -> "7", "000" -> "0".
string stripZeros(const string &s) {
    size_t i = 0;
    while (i + 1 < s.size() && s[i] == '0') i++;
    return s.empty() ? "0" : s.substr(i);
}

void reverseStr(string &s) {
    for (int i = 0, j = (int)s.size() - 1; i < j; i++, j--) {
        char t = s[i]; s[i] = s[j]; s[j] = t;
    }
}

// -1 if a < b, 0 if a == b, 1 if a > b.                                O(n + m)
int compareBig(const string &x, const string &y) {
    string a = stripZeros(x), b = stripZeros(y);
    if (a.size() != b.size()) return a.size() < b.size() ? -1 : 1;
    for (size_t i = 0; i < a.size(); i++)
        if (a[i] != b[i]) return a[i] < b[i] ? -1 : 1;
    return 0;
}

// a + b                                                                 O(n + m)
string addBig(const string &x, const string &y) {
    string a = stripZeros(x), b = stripZeros(y), res;
    int i = (int)a.size() - 1, j = (int)b.size() - 1, carry = 0;
    while (i >= 0 || j >= 0 || carry) {
        int sum = carry;
        if (i >= 0) sum += a[i--] - '0';
        if (j >= 0) sum += b[j--] - '0';
        res.push_back(char('0' + sum % 10));      // append (cheap), reverse once
        carry = sum / 10;
    }
    reverseStr(res);
    return res;
}

// a - b, requires a >= b                                               O(n + m)
string subBig(const string &x, const string &y) {
    string a = stripZeros(x), b = stripZeros(y), res;
    int i = (int)a.size() - 1, j = (int)b.size() - 1, borrow = 0;
    while (i >= 0) {
        int d = (a[i--] - '0') - borrow;
        if (j >= 0) d -= b[j--] - '0';
        if (d < 0) { d += 10; borrow = 1; } else borrow = 0;
        res.push_back(char('0' + d));
    }
    reverseStr(res);
    return stripZeros(res);
}

// a * b, schoolbook multiplication                                     O(n * m)
```

```
string mulBig(const string &x, const string &y) {
    string a = stripZeros(x), b = stripZeros(y);
    if (a == "0" || b == "0") return "0";
    int n = a.size(), m = b.size();
    string res(n + m, 0);                    // res[k] = digit VALUE at place 10^k
    for (int i = n - 1; i >= 0; i--) {
        int carry = 0, shift = n - 1 - i;    // shift = place value of a[i]
        for (int j = m - 1; j >= 0; j--) {
            int pos = shift + (m - 1 - j);   // place value of a[i] * b[j]
            int cur = res[pos] + (a[i] - '0') * (b[j] - '0') + carry;
            res[pos] = char(cur % 10);
            carry = cur / 10;
        }
        for (int pos = shift + m; carry; pos++) {
            int cur = res[pos] + carry;
            res[pos] = char(cur % 10);
            carry = cur / 10;
        }
    }
    for (int k = 0; k < n + m; k++) res[k] = char(res[k] + '0');
    reverseStr(res);
    return stripZeros(res);
}

// a / b (integer division, b != 0), long division                    O(n * m)
string divBig(const string &x, const string &y) {
    string a = stripZeros(x), b = stripZeros(y), q, rem = "0";
    for (size_t i = 0; i < a.size(); i++) {
        rem = (rem == "0") ? string(1, a[i]) : rem + a[i];   // bring down a digit
        int d = 0;
        while (compareBig(rem, b) >= 0) { rem = subBig(rem, b); d++; }   // d <= 9
        q.push_back(char('0' + d));
    }
    return stripZeros(q);
}
// ------------------------------------------------------------------------
```

1 · Custom Precedence Calculator

Task. The first line gives the precedence of `+ - * /` (higher binds tighter); the second line is an expression without spaces or parentheses. Operators of equal precedence are applied left to right, `/` is integer division. Print the value.

Idea: infix → postfix with a precedence table, then evaluate

This is the standard two-step method; the only change from normal C++ rules is that `precedence()` reads a table filled from the input instead of hard-coding "× and ÷ before + and −". That is exactly why the problem had custom precedence: a solution that hard-codes BODMAS, or evaluates "first all × ÷, then all + −", fails.

Step 1 (shunting-yard). Read tokens left to right. A number goes straight to the output. For an operator, first pop every operator on the stack whose precedence is **greater than or equal** to the new one, then push the new one. At the end, pop everything that is left.

Step 2. Scan the postfix: push numbers; for an operator pop **b first (right operand)**, then a, and push *a* op *b*. The single value left is the answer.

Worked example – Sample 2: precedences 3 1 2 2, expression `2+3*4-5`

Here `+` (3) binds tightest, then `*` and `/` (2), then `-` (1).

Token	Action	Output	Stack
`2`	number → output	`2`	
`+`	stack empty → push	`2`	`+`
`3`	number → output	`2 3`	`+`
`*`	top `+` has 3 ≥ 2 → pop it; push `*`	`2 3 +`	`*`
`4`	number → output	`2 3 + 4`	`*`
`-`	top `*` has 2 ≥ 1 → pop it; push `-`	`2 3 + 4 *`	`-`
`5`	number → output	`2 3 + 4 * 5`	`-`
end	pop the rest	`2 3 + 4 * 5 -`	

Evaluating `2 3 + 4 * 5 -`: (2 + 3) = 5, 5 × 4 = 20, 20 − 5 = **15** (with normal C++ rules the answer would be 9).

Why "≥" and not ">"?

Popping on *equal* precedence is what makes `10-3-2` mean (10−3)−2 = 5 and `100/10/5` mean (100/10)/5 = 2. With ">" equal operators are evaluated right to left: 10−(3−2) = 9 and 100/(10/5) = 50. Sample 3 (all precedences equal) is exactly this case.

Code

```
// Precedence of + - * / as given in the input (higher binds tighter).
int precPlus, precMinus, precMul, precDiv;
```

```
int precedence(char op) {
    if (op == '+') return precPlus;
    if (op == '-') return precMinus;
    if (op == '*') return precMul;
    return precDiv;
}

bool isDigit(char c) { return c >= '0' && c <= '9'; }
bool isOperator(char c) { return c == '+' || c == '-' || c == '*' || c == '/'; }

string applyOp(const string &a, const string &b, char op) {
    if (op == '+') return addBig(a, b);
    if (op == '-') return subBig(a, b);
    if (op == '*') return mulBig(a, b);
    return divBig(a, b);
}

// Step 1: infix -> postfix (shunting-yard), tokens separated by single spaces.
string infixToPostfix(const string &expr) {
    string out;
    stack<char> ops;
    size_t i = 0;
    while (i < expr.size()) {
        if (isDigit(expr[i])) {                      // whole number -> output
            size_t j = i;
            while (j < expr.size() && isDigit(expr[j])) j++;
            if (!out.empty()) out += ' ';
            out += expr.substr(i, j - i);
            i = j;
        } else {                                     // operator
            // Pop while the stack top has GREATER OR EQUAL precedence
            // ('>=' makes equal-precedence operators evaluate left to right).
            while (!ops.empty() && precedence(ops.top()) >= precedence(expr[i])) {
                out += ' ';
                out += ops.top();
                ops.pop();
            }
            ops.push(expr[i]);
            i++;
        }
    }
    while (!ops.empty()) {                           // flush remaining operators
        out += ' ';
        out += ops.top();
        ops.pop();
    }
    return out;
}

// Step 2: evaluate postfix with a stack of big integers.
string evalPostfix(const string &postfix) {
    stack<string> st;
    size_t i = 0;
    while (i < postfix.size()) {
        if (postfix[i] == ' ') { i++; continue; }
        size_t j = i;
        while (j < postfix.size() && postfix[j] != ' ') j++;
        string tok = postfix.substr(i, j - i);
        i = j;
        if (tok.size() == 1 && isOperator(tok[0])) {
            string b = st.top(); st.pop();           // RIGHT operand is popped first
            string a = st.top(); st.pop();
            st.push(applyOp(a, b, tok[0]));
        } else {
            st.push(stripZeros(tok));
        }
    }
```

```
    return st.top();
}

int main() {
    string expr;
    cin >> precPlus >> precMinus >> precMul >> precDiv >> expr;
    cout << evalPostfix(infixToPostfix(expr)) << "\n";
    return 0;
}
```

Running time

The conversion and the stack evaluation are O(number of tokens). All real cost is in the arithmetic: each operation is O(n + m) or O(n · m) on its operands. The reference solution takes at most 0.04 s on the hidden tests.

Mistakes seen in submissions

- **Zero printed as an empty line.** Removing leading zeros with `while (i < s.size() && s[i]=='0')` deletes every digit of 0, so `3/7` or `0*5` printed nothing.
- **">" instead of "≥"** when popping: equal-precedence operators come out right to left.
- **Operands popped in the wrong order:** the first value popped is the *right* operand. Swapped, `9 1 -` computes 1 − 9.
- **Forgetting to pop the remaining operators** at the end of the conversion.
- **Tokens running together:** without a space between tokens, `3+6` turns into the postfix `36 +`.
- **Ignoring the first input line** and always using C++ precedence.

2 · Big Integer Square Root

Task. Given N (up to 400 digits), print r = ⌊√N⌋ – the largest r with $r^2 \le N$ – and the remainder $N - r^2$. No floating point, no built-in `sqrt`.

Idea: binary search on the answer

"$r^2 \le N$" is true for small r and false for large r, so the largest r for which it holds can be found by binary search. Each step needs one addition and one division by 2 (for the middle), one multiplication and one comparison (the test), and one addition or subtraction (move a bound) – all five operations of the core.

- **Search range.** A d-digit N has a root of at most ⌈d/2⌉ digits, so r lies in $[0, 10^{\lceil d/2 \rceil}]$. Searching [0, N] is also correct, just about 3 times slower here.
- **Loop while lo ≤ hi**, not lo < hi: with "<" the last candidate is never tested.
- **Keep the best answer in its own variable** (`ans`), updated whenever $mid^2 \le N$. The last `mid` the loop tried is often one too large.
- **Remainder** = $N - r^2$, computed once after the search.

Worked example – Sample 1: N = 26

N has 2 digits, so the search range is [0, 10].

lo	hi	mid	mid^2	$mid^2 \le 26$?	Then
0	10	5	25	yes	ans = 5, lo = 6
6	10	8	64	no	hi = 7
6	7	6	36	no	hi = 5

Now lo = 6 > hi = 5, so the loop stops with **ans** = **5**; remainder 26 − 25 = **1**. Output `5 1`. Note that the last `mid` tried was 6, not the answer.

Code

```
// Largest r with r*r <= n, by binary search on [0, 10^ceil(d/2)].
// (A d-digit number has a square root of at most ceil(d/2) digits.)
string isqrtBig(const string &n) {
    string lo = "0";
    string hi = "1" + string((n.size() + 1) / 2, '0');
    string ans = "0";
    while (compareBig(lo, hi) <= 0) {                 // lo <= hi  (not lo < hi)
        string mid = divBig(addBig(lo, hi), "2");
        if (compareBig(mulBig(mid, mid), n) <= 0) {   // mid*mid <= n: mid works
            ans = mid;
            lo = addBig(mid, "1");
        } else {
            hi = subBig(mid, "1");                    // safe: mid >= 1 here
        }
    }
    return ans;
}

int main() {
    string n;
    cin >> n;
```

```
    n = stripZeros(n);
    string r = isqrtBig(n);
    cout << r << " " << subBig(n, mulBig(r, r)) << "\n";   // remainder = n - r^2
    return 0;
}
```

Running time

The range $[0, 10^{\lceil d/2 \rceil}]$ needs about $3.3 \cdot (d/2) \approx 670$ iterations for $d = 400$. Each iteration is dominated by the $O(d^2)$ multiplication of two ~200-digit numbers, so the total is $O(d^3)$. The reference solution takes 0.16 s on a 400-digit N.

Mistakes seen in submissions

- **`while (lo < hi)` instead of `lo ≤ hi`:** wrong for N = 1, 26, 100, …
- **Printing `mid` instead of `ans`** after the loop: often one too large (N = 2 gave `2 8`).
- **Remainder computed as N mod r^2** (divide, multiply back, subtract) instead of $N - r^2$. This is the same for $r \geq 3$ but wrong for N = 2, 3, 8 (N = 8 gave `2 0` instead of `2 4`).
- **Not leaving the loop when $mid^2 = N$:** printing the answer inside the loop without `return` / `break` made every perfect square (even 0 and 1) loop forever.
- **Upper bound too small for N = 1:** starting with hi = N − 1 or hi = N/2 gives 0 for N = 1, so the search never reaches the answer 1.
- **Testing mid ≤ N / mid instead of $mid^2 \leq N$:** mathematically fine (a habit from avoiding overflow with `long long`), but a full big division per step made it about 20 times slower. Big integers cannot overflow – just multiply.
- **Checking r = 0, 1, 2, … one by one** – about 10^{200} steps for 400 digits.

3 · Catalan Number

Task. Given $0 \le n \le 1000$, print $C(n) = (2n)! / ((n+1)! \cdot n!)$, computed with this factorial formula (compute the factorials as big integers, multiply the two in the denominator, and do one big division).

Idea

- Multiply $1 \cdot 2 \cdot 3 \cdot \ldots \cdot 2n$ in a single loop. On the way, when the loop reaches $i = n$ and $i = n + 1$, save the current product: those are $n!$ and $(n+1)!$. At the end the product is $(2n)!$.
- Use **separate `if` statements** to save them, not an `else if` chain: for $n = 1$, the values $n + 1$ and $2n$ are both 2, so one loop step must fill two variables.
- Multiply $n! \cdot (n+1)!$ and divide $(2n)!$ by it **once**.

Worked example – Sample 1: n = 5

The loop runs $i = 1 \ldots 10$ and saves $5! = 120$ at $i = 5$ and $6! = 720$ at $i = 6$; at the end $10! = 3628800$. Denominator $720 \times 120 = 86400$, and $3628800 / 86400 = \mathbf{42}$.

How big do the numbers get? (n = 1000)

Value	Digits
$(2n)! = 2000!$	5,736
$n! \cdot (n+1)! = 1000! \cdot 1001!$	5,139
$C(1000)$ – the answer	598

Long division does one round per digit of the dividend, but how much each round costs depends on the size of the divisor and the quotient. Dividing once by the 5,139-digit denominator produces only 598 quotient digits. Dividing twice – first $(2n)!/n!$, then by $(n+1)!$ – creates a 3,168-digit intermediate result and is several times slower.

Code

```
int main() {
    int n;
    cin >> n;

    // One pass 1, 2, ..., 2n; remember n! and (n+1)! on the way.
    // Separate ifs (not else-if): for n = 1, n+1 == 2n == 2.
    // For n = 0 the loop does not run and 0! = 1! = 1 are already correct.
    string fact = "1", factN = "1", factN1 = "1";
    for (int i = 1; i <= 2 * n; i++) {
        fact = mulBig(fact, to_string(i));
        if (i == n) factN = fact;
        if (i == n + 1) factN1 = fact;
    }
    string fact2N = fact;                    // (2n)!

    // C(n) = (2n)! / ((n+1)! * n!)  -- one multiplication, one big division.
    cout << divBig(fact2N, mulBig(factN1, factN)) << "\n";
    return 0;
}
```

Running time

Building (2n)! is 2n multiplications of a number of up to L ≈ 5,736 digits by a number of at most 4 digits: O(n · L). The final multiplication and division are O(L^2). The reference solution takes about 0.3 s for n = 1000.

Mistakes seen in submissions

- **`else if` chain when saving the factorials:** for n = 1 the step i = 2 is both 2n and n + 1, so (n+1)! was never saved. The denominator became an empty string and the division looped forever.
- **Building strings by prepending** (`ans = digit + ans`) in add/subtract: O(n^2) per operation. n = 1000 took 1.9 s instead of 0.3 s.
- **Dividing twice** ((2n)!/n!, then /(n+1)!): huge intermediate quotient, much slower.
- **Choosing the shorter multiplier with `a > b` on strings:** that compares alphabetically, not by size – `"12345…"` (3000 digits) counts as smaller than `"1500"`. Compare by length first (use your compare function).
- **Not following the required formula** (e.g. the recurrence C(k+1) = C(k)·2(2k+1)/(k+2), or cancelling (n+1)! in advance). These give correct output but were not the method the question asked for.
- **Removing all zeros** in the strip function: `"0"` became `""`. Most programs survived this only by luck; together with the `else if` bug above it is what turned the denominator into an empty string.

> **Testing tip for next time**
>
> Before submitting, always try the smallest inputs by hand: N = 0, 1, 2, 3, 8 for the square root, n = 0, 1, 2 for Catalan, a single number and results equal to 0 for the calculator. Almost every wrong answer in this lab was on one of these.